

1.1 Introduction

The rapid growth of the Internet of Things (IoT) has significantly changed the design and operation of modern surveillance systems. IoT technologies have enabled physical devices to communicate, exchange data, and provide remote services through network connectivity. Among the most important IoT devices used in homes, enterprises, and industrial environments are Internet Protocol (IP) cameras, which support real-time monitoring, remote access, and automated management functions [1][2][9][10].

Unlike traditional analog cameras, IP cameras operate as networked digital systems that can transmit video streams over local area networks and, in many cases, through Internet-connected services. These devices often support protocols and standards such as Real-Time Streaming Protocol (RTSP), Real-time Transport Protocol (RTP), Hypertext Transfer Protocol (HTTP), and Open Network Video Interface Forum (ONVIF), allowing them to integrate easily with video management systems and other smart security platforms. Although these features improve usability and scalability, they also increase the attack surface of the devices and create additional security concerns [2][3][6][7].

One of the major security challenges in IP camera deployments is that many organizations and users assume that internal local networks are inherently trusted. segmentation, strict access control, or continuous monitoring[3][4][18][19]. In such conditions, an attacker who gains local network access may conduct reconnaissance activities, enumerate devices, and intercept traffic without the need to exploit complex firmware vulnerabilities [1][4][5].

Furthermore, among the practical Local Area Network (LAN) -based threats that can affect IP camera systems are ONVIF-based device discovery and Address Resolution Protocol (ARP) spoofing attacks. ONVIF discovery mechanisms simplify camera installation and interoperability, but they may also reveal device metadata and service endpoints to unauthorized hosts within the same subnet. Similarly, ARP spoofing attacks exploit the lack of authentication in the Address Resolution Protocol to manipulate traffic flows and establish a Man-in-the-Middle position between communicating devices [6][7] [9][12].

For this reason, studying network-level indicators of these attacks is important for improving security monitoring in IoT-based surveillance environments. Therefore, this project focuses on evaluating the security impact of ONVIF enumeration and ARP spoofing in IP camera LAN environments and on designing a network-based intrusion detection approach capable of identifying suspicious traffic patterns generated during these attack scenarios [1][11][12][14].

1.2 Background and History

The development of surveillance technology has passed through several stages, beginning with traditional analog Closed-Circuit Television (CCTV) systems. Early surveillance solutions relied on analog cameras connected to dedicated recording equipment through closed cabling infrastructures. Although these systems provided basic monitoring functions, they lacked flexibility, scalability, and remote management capabilities [2][10].

With the expansion of digital networking technologies, surveillance systems gradually evolved toward Internet Protocol (IP)-based architectures. IP cameras introduced major improvements by allowing digital video transmission over standard network infrastructures. This shift enabled higher image quality, remote viewing, centralized management, and easier integration with software platforms and storage systems. As a result, IP cameras became widely adopted in residential, commercial, and industrial security environments [2][6].

Furthermore, the increasing integration of IP cameras with IoT ecosystems further expanded their functionality. Modern IP cameras are no longer simple recording devices; they now support motion detection, cloud connectivity, mobile access, event notifications, and interoperability with other smart systems. Standards such as ONVIF were introduced to improve compatibility between devices from different vendors by defining common discovery, configuration, and media service mechanisms [6][7][10].

However, this technological progress has also introduced new cybersecurity risks. As cameras became connected to local networks and remote services, they began to expose web interfaces, streaming endpoints, and device discovery mechanisms that could be abused by attackers. In many deployments, security configurations have not advanced at the same pace as functionality, leading to weak authentication practices, poor network segmentation, and limited monitoring capabilities [1][4][5].

Furthermore, while earlier IoT security research primarily focused on Internet-facing attacks and firmware vulnerabilities, more recent studies highlight the growing importance of internal network threats. In IP camera deployments, attacks such as unauthorized ONVIF enumeration and ARP spoofing can be launched by adversaries already present within the LAN, making internal visibility and traffic-based detection increasingly important for maintaining system security [6][9][11][12][21].

1.3 Problem Statement

Although IP cameras are commonly deployed within internal networks, many installations lack proper segmentation and monitoring. Features such as ONVIF-based automatic discovery simplify device integration but may unintentionally expose device information to unauthorized internal entities. Additionally, local network attacks such as ARP spoofing can enable traffic interception without directly exploiting firmware vulnerabilities [4][5][6][7].

The absence of continuous network-based intrusion detection in many IoT environments allows such attacks to remain unnoticed [11][12]. Therefore, there is a need to analyze how reconnaissance and interception attacks can be executed in a controlled LAN environment and how observable traffic indicators can support their detection. This lack of visibility into internal network activity makes it difficult for administrators to detect early-stage reconnaissance and interception attempts. Without proper monitoring mechanisms, such attacks may remain undetected for long periods, increasing the risk of data exposure and unauthorized access within IP camera environments [5][6][18][19].

1.4 Aim and Objectives

The aim of this project is to evaluate the security impact of ONVIF-based device enumeration and ARP spoofing attacks in IP camera LAN environments and to design a network-based intrusion detection approach capable of identifying these attack behaviors. The objectives of this project are

- To Analyze the security risks associated with IP camera environments, including ONVIF discovery mechanisms and network traffic exposure.
- To Capture and evaluate network traffic behavior during normal operation and security-related events, including ARP spoofing attacks and traffic interception risks.
- To Design and implement a Network Intrusion Detection System (NIDS) approach using detection rules to identify suspicious activities and abnormal network behavior.
- To Evaluate mitigation strategies and automated response mechanisms to reduce security risks and improve the overall security of IP camera deployment

1.5 Literature Review Overview

Recent research in IoT and IP camera security has identified multiple vulnerabilities associated with networked surveillance systems, particularly in environments where security controls are limited. Studies indicate that IP cameras are frequently targeted due to their continuous operation, exposure to network services, and ability to transmit sensitive visual data [1][9][10].

Furthermore, several researchers have explored ONVIF-based discovery mechanisms as a potential security risk.

Although ONVIF simplifies device integration, it allows any host within the same network to enumerate devices and retrieve metadata such as IP addresses, service endpoints, and supported capabilities [6][7]. This information can significantly assist attackers in planning further exploitation or interception attacks.

In addition to reconnaissance techniques, ARP spoofing has been widely studied as a practical attack method in local networks. Due to the lack of authentication in ARP, attackers can manipulate address mappings and position themselves as intermediaries between communicating devices. In IP camera environments, this may allow interception of RTSP streams and analysis of traffic patterns without direct access to the camera itself [9][12]. Furthermore, modern intrusion detection research emphasizes the importance of network-based monitoring systems in IoT environments.

Since many IoT devices have limited processing power and lack advanced security features, Network Intrusion Detection Systems (NIDS) are considered an effective approach for identifying suspicious traffic patterns and abnormal behavior [11][12][14]. However, despite these contributions, there remains a gap between theoretical research and practical implementation in controlled environments. Therefore, this project addresses this gap by combining real attack simulations with traffic analysis and IDS rule design, providing a more applied perspective on detecting LAN-based threats in IP camera systems [1][11][12].

1.6 Research Organization (Report Structure)

Chapter I – Introduction: Introduces the project, defines the problem, and presents the objectives and scope.

Chapter II – Literature review strategy: Explains the research methodology used in this project, including the IMGSIE approach, experimental lab setup, tools, attack simulation procedures, traffic capture, analysis, IDS rule design, and detection evaluation

Chapter III – Internet of things (LOT): Provides IoT and IP camera security theory, architectures, attack surfaces, and LAN-based threats.

Chapter IV – Theoretical Attack Model and IDS Design: Describes LAN threat model, ONVIF enumeration, ARP poisoning, network-based IDS design, detection logic, observable indicators, and behavioral detection strategy.

Chapter V –Experimental Implementation and Results: Shows practical execution of attacks, traffic capture and analysis, IDS alerts, and examples of observed patterns.

Chapter VI – Conclusion, Limitations and future work: Summarizes findings, evaluates IDS effectiveness, proposes mitigation strategies, and suggests future work including possible Intrusion Prevention System (IPS) integration.

1.7 Project Scope

This project focuses exclusively on LAN-based attacks against IP camera environments. The study includes:

- ONVIF discovery and enumeration within a local subnet.
- ARP spoofing for traffic interception between devices.
- Network-level traffic monitoring using IDS mechanisms.

The project does not include Internet-wide scanning, firmware exploitation (e.g., CVE-based attacks), remote code execution, or destructive denial-of-service scenarios. All simulations are conducted within an isolated and authorized laboratory environment for academic purposes only.

2.1 Methodology

This research follows the IMGSIE methodology, which stands for Introduction, Method, Generalization, Specification, Implementation, and Evaluation. The methodology provides a structured framework for organizing the research process and guiding the project from problem definition through implementation and final evaluation. Within this framework, an experimental and analytical approach was employed to assess security threats targeting IP camera environments in local area networks (LANs). The study was conducted in a controlled laboratory environment to safely simulate realistic attack scenarios and observe their impact on network behavior without affecting real-world systems [1][2][3].

The research focuses on two representative attack techniques: ONVIF-based device enumeration and ARP spoofing. These attacks were selected because they represent practical threats commonly encountered in IoT environments and exploit protocol behavior rather than software vulnerabilities [4][6][7][9][21]. The experimental environment consists of an attacker machine, an IP camera system, and a client device connected within an isolated network. The attacker system performs reconnaissance and interception activities, while the remaining devices simulate normal communication patterns. This controlled setup enables accurate comparison between normal traffic and attack-generated anomalies [11][12].

Network traffic is monitored and captured using Wireshark to observe packet-level activity during both normal operation and attack execution. The collected traffic is analyzed to identify indicators of malicious behavior, including multicast discovery messages and abnormal ARP responses. Based on these observations, detection mechanisms are implemented using the Suricata Network Intrusion Detection System (NIDS), where custom rules are designed to identify abnormal ONVIF discovery activity and ARP-related anomalies in real time [12][22].



Figure 2.1: IMGSIE research methodology adopted in this study.

2.2 Research Life Cycle

The research life cycle adopted in this study consists of eight sequential phases. The process begins with environment setup, where an isolated laboratory network is prepared for experimentation. This is followed by reconnaissance activities using Nmap and ONVIF-based discovery to identify available devices and services. The study then proceeds to RTSP observation and ARP spoofing simulation in order to analyze potential security weaknesses within the IP camera environment. Network traffic is subsequently captured and analyzed using Wireshark, while Suricata IDS is employed to monitor suspicious activities and generate security alerts [6][7][22]. In the mitigation phase, Fail2ban is used to implement automated response actions against detected threats. Finally, the effectiveness of the proposed detection and mitigation mechanisms is evaluated. Figure 2.2 illustrates the complete workflow followed in this study.

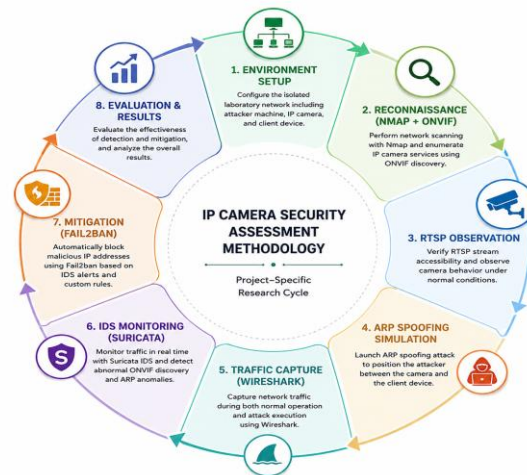


Figure 2.2 Project-Specific research methodology cycle for IP camera security assessment

2.3 Literature Review Strategy

A structured literature review was conducted to support the selection of attack scenarios and detection techniques. Academic databases such as IEEE Xplore, SpringerLink, ACM Digital Library, and Google Scholar were consulted.

Primary keywords included:

- “IP camera security”
- “ONVIF discovery vulnerability”
- “ARP poisoning LAN attack”
- “RTS P interception”
- “IoT network-based IDS”

Studies were selected if they:

- Analyzed practical IP camera vulnerabilities.
- Demonstrated LAN-based attack techniques.
- Described traffic-based detection indicators.

Studies unrelated to network-level IoT security or lacking technical validation were excluded.

2.3.1 Literature Review Analysis and Discussion

Recent research highlights that many critical threats against IP cameras originate inside local networks rather than from external Internet exposure[1][9][10]. Attacks such as ONVIF-based enumeration and ARP spoofing enable device discovery and traffic interception even when Internet access is restricted[6][7][21]. Traditional vulnerability-based approaches (e.g., CVE scanning) identify known weaknesses but do not fully capture real-time attack behavior observable in network traffic. Modern security research increasingly emphasizes behavioral and traffic-based monitoring using IDS systems[11][12][14][22]. This project addresses the gap between theoretical vulnerability discussion and practical LAN-based experimentation by implementing controlled attack simulations and analyzing corresponding network indicators.

2.4 Ethical Considerations

All simulations are conducted within an isolated and authorized laboratory environment. No real-world surveillance systems are targeted. The project is strictly for educational and defensive research purposes and promotes responsible security practices.

2.5 Experimental Environment, Tools, and Implementation Setup

The practical implementation of this project is conducted within a fully isolated local area network (LAN) to simulate a realistic IP camera deployment scenario. The environment consists of two main systems:

- An attacker machine running **Kali Linux**.
- A real Hikvision IP camera deployed within the isolated LAN environment.
- The simulation focuses exclusively on ONVIF enumeration and ARP spoofing attacks.

Software and Tools Used

The following tools are utilized during the experimental phase:

- **Kali Linux** – Used as the attacker platform to execute ONVIF enumeration and ARP spoofing attacks.
- **Hikvision IP Camera** – Used as the target surveillance device supporting RTSP and ONVIF services [6][7].
- **Ettercap / Arpspoof** – Used to perform ARP poisoning and establish a Man-in-the-Middle (MITM) position.
- **Wireshark** – Used to capture and analyze network traffic generated during normal and attack conditions.
- **Suricata** – Configured as a Network-Based Intrusion Detection System (IDS) to monitor traffic and generate alerts based on predefined detection rules [24].
- **Arpwatch** – Used to monitor ARP table changes and detect suspicious Media Access Control (MAC)/IP mapping anomalies during ARP spoofing attacks.
- **Fail2ban** – Used to implement automated response actions by dynamically blocking suspicious traffic detected through IDS alerts.
- **VLC** – Used to verify RTSP stream accessibility during interception testing.

Implementation Overview

The experiment follows this sequence:

1. Deploy the Hikvision IP camera with ONVIF enabled inside the isolated LAN.
2. Perform ONVIF enumeration from the attacker machine to identify exposed device information.
3. Launch ARP spoofing to position the attacker between the camera and the client device.
4. Capture network traffic using Wireshark during both normal operation and attack scenarios.
5. Configure and validate Suricata IDS rules to detect abnormal ONVIF discovery patterns and ARP anomalies.

3.1 Evolution of the Internet of Things

The Internet of Things (IoT) represents a fundamental shift in how computing systems interact with the physical world. Initially emerging from machine-to-machine (M2M) communication concepts, IoT has evolved into a global infrastructure connecting billions of heterogeneous devices. Advances in embedded systems, wireless communication, and cloud computing have accelerated this evolution, enabling real-time data collection and automation across diverse environments [1][8][10][15].

Despite these advancements, the rapid expansion of IoT has outpaced the development of robust security frameworks. Many IoT systems were originally designed with functionality and scalability as primary goals, leaving security as a secondary concern. This historical imbalance continues to influence modern IoT deployments, where legacy design decisions introduce persistent vulnerabilities [2][9].

3.2 IoT Ecosystems and Deployment Domains

IoT ecosystems encompass a wide range of deployment domains, each with unique operational requirements and threat profiles. These domains include:

- Consumer IoT, which includes smart home devices, wearables, and personal monitoring systems
- Industrial IoT, which focuses on manufacturing, energy, and critical infrastructure
- Enterprise IoT deployments, which integrate sensors and actuators into business operations and often interface with cloud services and third-party platforms

These diverse domains demonstrate the widespread adoption of IoT technologies across modern digital environments [1][10]. However, these heterogeneous environments introduce significant security challenges due to differences in device capabilities, ownership models, and regulatory requirements. A single IoT ecosystem may include devices from multiple vendors operating under inconsistent security standards, which complicates the enforcement of uniform protection mechanisms. This lack of standardization increases the difficulty of maintaining visibility, control, and accountability across the system, ultimately expanding the overall attack surface of IoT deployments [2][4][9].

3.3 IoT Architecture Models

IoT architectures are commonly structured using layered or distributed models that separate sensing, communication, processing, and application functions. Typical architectures include:

- **Perception** layers responsible for data acquisition.
- **Network** layers that enable data transmission.
- **Application** layers that provide user-facing services.
- More advanced **implementations** may also incorporate.
- Intermediate gateways.
- Edge **computing** nodes to improve scalability, reduce latency, and enhance data processing efficiency.

These architectural models support the integration of heterogeneous devices while maintaining functional modularity across the system [1][10]. However, while architectural layering improves interoperability and system organization, it also introduces multiple security challenges. Data must traverse several layers and components before reaching its destination, creating opportunities for interception, manipulation, or disruption at different points within the system. Each layer may expose distinct vulnerabilities, and weaknesses in one layer can propagate across the entire architecture. Therefore, understanding IoT architectural models is essential for identifying potential attack vectors and evaluating how security risks can spread within complex IoT environments[2][9].

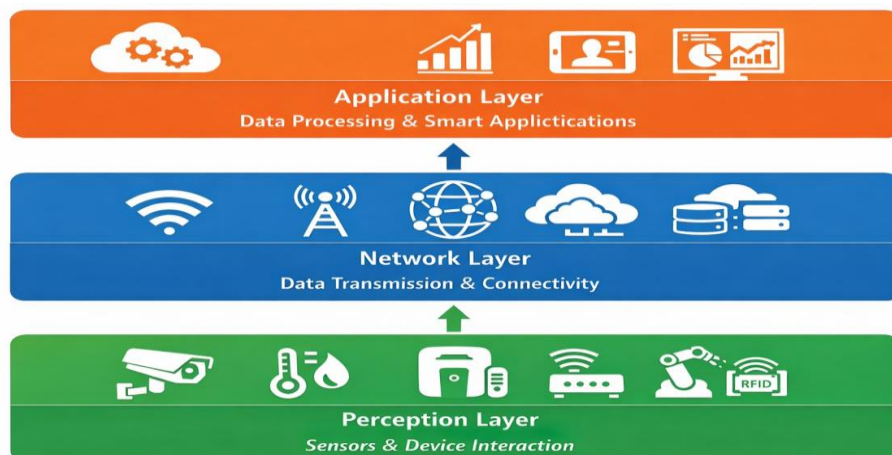


Figure 3.1: IoT Layered Architecture

3.4 Communication Technologies and Protocol Diversity

IoT communication relies on a diverse set of technologies and protocols designed to support varying bandwidth, latency, and power constraints. These include traditional IP-based protocols such as HTTP and Transmission Control Protocol/Internet Protocol (TCP/IP), as well as application-layer protocols tailored for constrained environments. In IP camera systems, communication commonly involves protocols such as RTSP for real-time video streaming, RTP for media transport, and HTTP for device management and configuration. The coexistence of multiple protocols within a single IoT deployment reflects the need for flexibility and interoperability across heterogeneous devices [1][6][7].

However, this protocol diversity introduces additional security challenges. Each communication protocol exposes different interfaces and potential vulnerabilities, increasing the overall attack surface of the system. Furthermore, many IoT protocols prioritize efficiency and low resource consumption over strong security mechanisms, often relying on simplified or optional protection features. This imbalance creates conditions where attackers can exploit protocol behavior rather than software vulnerabilities, particularly in local network environments where trust assumptions are weak [2][4][5].

3.5 Security Challenges in Large-Scale IoT Systems

Securing IoT systems presents unique challenges that differ significantly from traditional IT environments. Many IoT devices are constrained in terms of processing power, memory, and energy resources, which limits the implementation of strong cryptographic mechanisms and advanced security controls. Additionally, large-scale deployments often involve thousands of distributed devices, making centralized management, monitoring, and patching difficult to maintain. These factors contribute to an environment where vulnerabilities can persist for long periods without detection [2][5][8][9].

Furthermore, IoT devices typically have long operational lifecycles and may remain in use even after vendor support has ended. This increases the risk of unpatched vulnerabilities and outdated security mechanisms being exploited by attackers. In many cases, devices operate unattended, reducing the likelihood that abnormal behavior will be noticed in a timely manner. As a result, attackers may gain prolonged access to systems or conduct malicious activities with minimal resistance, highlighting the need for effective network-level monitoring and intrusion detection strategies [1][5][12][17].

3.6 IoT Attack Surface and Exposure Complexity

The attack surface of an IoT system encompasses all hardware, software, and network interfaces through which an attacker may gain access. Unlike traditional computing environments, IoT attack surfaces are highly dynamic, expanding as new devices are added, updated, or reconfigured within the network. Key exposure points may include Remote management interfaces Wireless communication channels Cloud integrations These components further increase system exposure, creating multiple entry points that may be exploited by adversaries [2][5][8][9].

Moreover, the distributed and heterogeneous nature of IoT deployments makes it difficult to achieve complete visibility over all connected components. Devices may operate under different configurations and security policies, leading to inconsistencies in protection levels across the system. As a result, misconfigurations, weak access controls, and unintended exposures may remain unnoticed, increasing the likelihood of successful attacks. These complexities highlight the importance of continuous monitoring and comprehensive security strategies to manage evolving IoT attack surfaces [1][2][5].

3.7 Monitoring, Visibility, and Security Limitations in IoT

Effective monitoring is a critical component of cybersecurity, yet it remains particularly challenging in IoT environments. Many IoT devices generate continuous and predictable traffic patterns, which can make subtle anomalies difficult to detect using traditional monitoring approaches. In addition, limited logging capabilities and constrained computational resources reduce the ability of individual devices to perform local security analysis or maintain detailed activity records [2][5][9].

Furthermore, the lack of centralized visibility across heterogeneous IoT deployments complicates the detection of abnormal behavior at the system level. Devices from different vendors may operate with inconsistent configurations, making it difficult to establish a unified monitoring strategy. These limitations highlight the importance of network-level visibility, where aggregated traffic analysis can provide a more comprehensive view of system behavior. As a result, network-based intrusion detection systems (NIDS) become essential for identifying suspicious patterns and compensating for the inherent security limitations of IoT devices [5][11][12].

3.8 IP Cameras as IoT Devices in LAN Environments

Internet Protocol (IP) cameras represent one of the most widely deployed categories of IoT devices within modern local area networks. Unlike traditional analog surveillance systems, IP cameras operate as fully networked endpoints that

transmit digital video streams over IP-based infrastructures. These devices typically support real-time streaming protocols such as RTSP and often implement standardized discovery and management frameworks such as ONVIF to facilitate interoperability and integration across different platforms [6][7].

Within enterprise and residential environments, IP cameras are commonly deployed inside internal LAN networks under the assumption that internal communication is inherently trusted. As a result, minimal segmentation, weak access control policies, and limited traffic monitoring are frequently observed in such deployments [4][5][18][19]. Cameras may expose web-based management interfaces, streaming endpoints, and automatic discovery services that are accessible to any device within the same subnet. While these features enhance usability and centralized management, they also significantly expand the internal attack surface [2][4][5].

If an unauthorized entity gains access to the local network, the absence of strict segmentation and monitoring mechanisms can enable device reconnaissance and traffic interception without requiring exploitation of firmware vulnerabilities. This makes IP camera environments particularly vulnerable to protocol-based attacks that rely on trust assumptions within LAN infrastructures [4][5][21].

3.9 LAN-Based Threats Against IP Camera Systems

Security risks affecting IP camera deployments are not limited to Internet-facing exposures. In many practical scenarios, significant threats originate from within the local network itself. Internal adversaries, compromised hosts, or unauthorized devices connected to the same subnet may exploit protocol behaviors and weak trust assumptions to conduct reconnaissance and interception activities. Unlike external attacks, these threats do not necessarily rely on exploiting software vulnerabilities but instead leverage legitimate network functionality in unintended ways [4][5][9].

One common reconnaissance technique involves automatic device discovery mechanisms that reveal camera metadata and service endpoints. Standards such as ONVIF simplify deployment and interoperability but may unintentionally expose device information to any host capable of sending broadcast-based discovery requests. This type of enumeration enables attackers to identify active devices, retrieve configuration details, and prepare further attack stages without requiring authentication or direct access to the device itself [6][7].

In addition to reconnaissance, local network attacks such as ARP spoofing allow adversaries to manipulate traffic flows by altering IP-to-MAC address mappings between communicating devices. By positioning themselves as

intermediaries within the communication path, attackers can intercept, observe, or potentially modify traffic exchanged between cameras and client systems. These attacks highlight the weaknesses of trust-based LAN architectures and demonstrate how protocol-level manipulation can lead to serious security implications even in the absence of firmware exploitation [4][21].

The combination of automatic discovery exposure and address resolution manipulation underscores the importance of network-level visibility and intrusion detection. Since many IP camera environments lack continuous internal monitoring, reconnaissance and interception activities may remain undetected without dedicated network-based security mechanisms. This reinforces the need for behavioral traffic analysis and IDS-based detection approaches, which form the foundation for the threat modeling and detection Strategies discussed in the following chapter [5][11][12][22].

4.1 LAN Threat Model for IP Camera Environments

Unlike Internet-facing attacks, many realistic threats against IP cameras originate from within local area networks (LANs). In both enterprise and residential deployments, cameras are often installed under the assumption that internal networks are trusted environments. This trust model reduces the implementation of strict access control and continuous monitoring mechanisms, increasing the risk of internal reconnaissance and interception activities [4][5][9].

In a typical LAN-based scenario, an attacker may already have network access through a compromised host, weak wireless security, or unauthorized physical connection. Without exploiting firmware vulnerabilities, the attacker can leverage standard protocol behaviors to gather device information or intercept communication traffic. This project focuses specifically on such internal, protocol-level threats, emphasizing realistic attack conditions that reflect common deployment environments [4][6][21].

4.2 ONVIF Discovery as a Reconnaissance Vector

ONVIF (Open Network Video Interface Forum) provides standardized mechanisms for automatic discovery and management of IP cameras within local networks. Using Web Services Dynamic Discovery (WS-Discovery), ONVIF-enabled devices broadcast their presence to simplify integration and configuration. While this functionality improves deployment efficiency, it may also be misused by unauthorized entities to perform device enumeration and extract sensitive metadata [6][7].

An attacker capable of sending discovery requests within the local subnet may obtain information such as device IP addresses, supported services, streaming endpoints, and manufacturer details. This process does not require authentication or exploitation of firmware vulnerabilities, as it relies on legitimate protocol behavior. As a result, ONVIF discovery represents a practical reconnaissance technique that expands the internal attack surface of IP camera deployments [6][7][20].

From a detection perspective, ONVIF enumeration generates identifiable multicast traffic patterns that can be monitored at the network level. Repeated discovery requests or abnormal bursts of multicast messages may indicate reconnaissance activity, making this behavior suitable for detection using network-based intrusion detection systems [11][12][22].

4.2.1 ONVIF Service Architecture and Operational Model

ONVIF is based on a service-oriented architecture that utilizes web service technologies to enable interoperability between devices. Communication is typically

implemented using Simple Object Access Protocol (SOAP) messages transmitted over HTTP or Hypertext Transfer Protocol Secure (HTTPS). Each ONVIF-compliant device exposes structured services that can be queried by clients within the same network segment [6][7].

The core ONVIF services relevant to IP camera environments include:

- Device services.
- Media services.
- Imaging services.
- Event services.

These services provide access to configuration settings, streaming profiles, camera parameters, and event notifications. During enumeration, an unauthorized client may query these services to extract metadata and identify available functionalities, including RTSP streaming endpoints [6][7][20]. Because ONVIF responses are structured and standardized, enumeration can yield consistent and machine-readable information. This enables attackers to automate device discovery and mapping processes within a network, increasing the efficiency of reconnaissance activities [6][7].

4.2.2 WS-Discovery Mechanism and Message Structure

ONVIF discovery relies on WS-Discovery, a multicast-based protocol designed for automatic device detection within local networks. WS-Discovery operates over User Datagram Protocol (UDP) using multicast address 239.255.255.250 and port 3702. During the discovery process, a client sends a Probe message to the multicast group, and ONVIF-enabled devices respond with ProbeMatch messages containing device information [6][7].

These responses typically include:

- Device service Endpoint Reference (EPR).
- Service access Uniform Resource Locators (URLs) Transport Address (XAddr) .
- Supported scopes.
- Device type information.

Because discovery messages are broadcast within the subnet, any host capable of generating probe requests can trigger responses from all ONVIF-enabled devices

without prior knowledge of their IP addresses [6]. From a network analysis perspective, WS-Discovery generates identifiable multicast UDP traffic patterns containing Extensible Markup Language (XML)-based SOAP payloads. These characteristics make ONVIF enumeration observable and suitable for detection using packet inspection and traffic analysis techniques [11][12].

4.2.3 Security Implications of ONVIF Enumeration

The ONVIF enumeration process does not inherently exploit a software vulnerability; instead, it leverages standard protocol functionality. However, security risks arise due to the accessibility of discovery mechanisms to all hosts within the local subnet. Service metadata may reveal detailed information about device capabilities, configurations, and streaming endpoints, which can be used by attackers to plan further actions [6][7][20].

In environments without strict network segmentation, enumeration enables rapid identification of deployed devices and reduces the effort required to prepare interception or exploitation attempts. Even when authentication is enforced for configuration changes, metadata disclosure during discovery may still expose valuable information. These characteristics highlight the importance of monitoring discovery activity as part of a comprehensive security strategy [4][5][6].

From a detection perspective, multicast discovery bursts and structured response patterns create measurable network artifacts that can be used to identify abnormal behavior. This makes ONVIF enumeration a suitable candidate for behavioral detection using network-based intrusion detection systems [11][12][22].

4.3 ARP Poisoning as a Traffic Interception Technique

The Address Resolution Protocol (ARP) is responsible for mapping IP addresses to MAC addresses within a local network. ARP operates without authentication mechanisms, which makes it vulnerable to spoofing attacks. In a typical ARP poisoning scenario, an attacker sends forged ARP reply messages to convince network devices that the attacker's MAC address corresponds to another legitimate host, such as the default gateway or an IP camera [21].

As a result, traffic between communicating devices can be redirected through the attacker, allowing the attacker to establish a Man-in-the-Middle (MITM) position within the network. In IP camera environments, this enables interception of RTSP streams, observation of session establishment, and analysis of communication patterns without requiring direct access to the camera device or exploitation of firmware vulnerabilities.

Unlike exploit-based attacks, ARP poisoning relies on weaknesses in protocol design and trust assumptions within local networks. Since ARP responses are accepted without verification, devices may update their ARP cache based on malicious replies, allowing attackers to manipulate communication paths. This makes ARP poisoning a practical and effective technique for traffic interception in LAN-based environments [5][21].

4.3.1 ARP Protocol Structure and Operational Behavior

The Address Resolution Protocol (ARP) operates at the boundary between the data link layer and the network layer in Internet Protocol version 4 (IPv4) -based networks. Its primary function is to resolve IP addresses into corresponding MAC addresses to enable communication within the same local subnet. ARP messages contain key fields such as sender and target IP addresses, MAC addresses, and operation types indicating whether the message is a request or a reply [21]. When a host needs to communicate with another device, it broadcasts an ARP request asking for the MAC address associated with a specific IP address.

The device owning that IP responds with an ARP reply containing its MAC address, which is then stored in the ARP cache of the requesting host. This process allows efficient communication without repeated address resolution requests. However, ARP lacks built-in authentication or integrity verification mechanisms. Devices accept ARP replies regardless of whether they previously sent a request, which creates a fundamental security weakness. This behavior enables attackers to introduce forged ARP messages into the network and manipulate address mappings between devices [5][21].

4.3.2 ARP Cache Poisoning Mechanism

ARP cache poisoning exploits the absence of verification in ARP communication. An attacker sends forged ARP reply packets to one or more devices within the network, falsely associating the attacker's MAC address with the IP address of another legitimate host. This can include impersonating the default gateway or the IP camera itself [5][21].

In a typical attack scenario, the attacker sends spoofed ARP replies to both the camera and the gateway. Each device updates its ARP cache with the attacker's MAC address, believing it to be the legitimate communication partner. As a result, traffic between the two devices is redirected through the attacker, allowing interception and potential manipulation of data [21]. To maintain control over the communication path, the attacker must continuously send forged ARP replies at regular intervals, as ARP cache entries may expire or be updated. This repeated

activity generates observable traffic patterns that can be analyzed for detection purposes [11][12].

4.3.3 Traffic Characteristics of ARP Poisoning

ARP poisoning produces distinct traffic patterns that differ from normal network behavior. In stable environments, ARP traffic is typically limited and occurs only when address resolution is required. However, during an attack, abnormal ARP activity becomes noticeable due to repeated and unsolicited communication [21].

Common indicators of ARP poisoning include:

- Unsolicited ARP reply packets
- Multiple ARP replies claiming ownership of the same IP address
- Rapid repetition of ARP updates from a single MAC address
- Inconsistent or frequently changing IP-to-MAC mappings

These anomalies can be detected through packet inspection and traffic analysis tools such as Wireshark or network-based intrusion detection systems [11][12][21].

4.3.4 Security Implications in Camera Deployments

In many IP camera deployments, internal network communication is assumed to be secure, and minimal protective mechanisms are implemented at the network level. Cameras are rarely configured with static ARP bindings or advanced protection features such as Dynamic ARP Inspection, which increases their exposure to ARP-based attacks [5][21].

The practical significance of ARP poisoning lies in its simplicity and effectiveness. The attack does not require privileged access to the camera or exploitation of software vulnerabilities. Instead, it relies solely on network presence within the same broadcast domain, making it a realistic threat in many deployment scenarios [21]. From a defensive perspective, the predictable structure of ARP traffic and the abnormal repetition of forged replies provide measurable indicators that can be used for detection. This highlights the importance of network-based monitoring and IDS mechanisms in identifying and mitigating such attacks in real-world environments [11][12][22].

4.4 Observable Network Indicators of LAN-Based Attacks

Both ONVIF enumeration and ARP poisoning generate distinct and observable network traffic characteristics that can be identified through packet

inspection and traffic analysis. Unlike exploit-based attacks, these techniques rely on protocol behavior, which produces measurable patterns within network communication [11][12][22]. ONVIF enumeration is typically associated with multicast-based WS-Discovery traffic, including repeated Probe requests and corresponding responses from multiple devices within a short time interval. This results in noticeable bursts of multicast UDP traffic, often containing structured SOAP messages. Such patterns are uncommon during normal operation and may indicate systematic device discovery activity [6][7][11].

Similarly, ARP poisoning produces abnormal Layer 2 traffic behavior. Indicators include repeated unsolicited ARP replies, conflicting IP-to-MAC mappings, and an unusual increase in ARP packet frequency compared to baseline traffic levels. These anomalies can be detected by analyzing ARP communication patterns over time [11][12][21]. These observable indicators form the foundation for behavioral detection strategies. Instead of relying on known exploit signatures, detection focuses on identifying deviations from normal traffic behavior, making it more effective in detecting protocol-based attacks within IoT environments.

4.5 Network-Based IDS Approach for LAN Attack Detection

Due to the limited security capabilities of many IoT devices, network-level monitoring provides a practical and effective approach for detecting malicious activity. A Network-Based Intrusion Detection System (NIDS) can analyze aggregated traffic and identify abnormal patterns that indicate potential attacks. This is particularly important in IP camera environments, where internal monitoring is often limited [11][12][22].

In this project, IDS rules are designed to detect abnormal ONVIF discovery behavior and ARP poisoning activity. Detection strategies focus on identifying excessive multicast discovery traffic, repeated enumeration attempts, and abnormal ARP reply patterns. By correlating these indicators with known attack behaviors, the IDS can generate alerts that reflect potential security incidents [12][22].

Unlike traditional signature-based detection methods, this approach emphasizes behavioral analysis. Since the attacks examined in this project do not rely on exploit payloads, detection is based on deviations in normal traffic patterns rather than predefined attack signatures. This increases the adaptability and effectiveness of the detection mechanism across different environments [11][12].

4.6 IDS Rule Logic and Detection Strategy Design

Effective detection of LAN-based attacks against IP camera environments requires translating protocol-level weaknesses into measurable traffic indicators.

Rather than relying on exploit signatures or firmware-specific vulnerabilities, this project adopts a behavior-based detection strategy using a Network Intrusion Detection System (NIDS). The detection logic is derived directly from the observable characteristics of ONVIF enumeration and ARP poisoning discussed in previous sections [11][12][22]

4.6.1 Detection Logic for ONVIF Enumeration

ONVIF discovery relies on WS-Discovery multicast traffic over UDP port 3702. Under normal operational conditions, discovery messages are infrequent and typically generated during device installation or configuration [6][7]. However, reconnaissance activity may exhibit the following abnormal characteristics:

- Multiple WS-Discovery Probe messages within short time intervals
- Repeated multicast discovery bursts from a single host
- Sequential enumeration attempts targeting all devices in the subnet
- From a detection standpoint, the IDS strategy includes:
 - Monitoring UDP traffic directed to multicast address 239.255.255.250
 - Inspecting SOAP-based Probe message structures

Applying threshold-based rules to identify excessive discovery frequency

Instead of flagging a single discovery packet as malicious, detection focuses on behavioral patterns indicating systematic enumeration [6][11].

4.6.2 Detection Logic for ARP Poisoning

ARP spoofing generates abnormal Layer 2 traffic patterns that differ from standard network behavior. In stable LAN environments, ARP replies occur mainly in response to legitimate requests [5][21].

- Poisoning attempts may produce:
 - Unsolicited ARP reply packets
 - Multiple ARP replies claiming ownership of the same IP address
 - Rapid repetition of ARP updates from a single MAC address
 - Inconsistent IP-to-MAC mapping changes within short time intervals

The IDS detection strategy includes:

- Monitoring ARP reply frequency

- Identifying duplicate IP address claims
- Detecting sudden changes in ARP cache mappings
- Applying threshold-based anomaly detection for excessive ARP traffic

Since ARP operates below traditional IP-layer filtering mechanisms, IDS visibility at the network interface level is essential for reliable detection [11][12][21].

4.6.3 Behavioral vs Signature-Based Detection

Traditional signature-based detection focuses on identifying known exploit payloads or malware patterns. However, the attacks analyzed in this project do not rely on exploit code. Instead, they leverage legitimate protocol behavior in unintended ways.

Therefore, this project emphasizes behavioral detection, where:

- Enumeration is identified by abnormal multicast discovery activity.
- MITM positioning is inferred from anomalous ARP traffic patterns.

This approach increases detection generalization because it is not dependent on specific tools or exploit implementations. Any tool generating similar traffic patterns can be detected using the same behavioral logic [11][12][14].

4.6.4 Detection Limitations and Practical Considerations

Although network-based IDS provides visibility into traffic anomalies, certain limitations must be acknowledged:

- Low-frequency enumeration may resemble legitimate configuration activity.
- Highly controlled ARP spoofing with minimal packet frequency may evade threshold-based detection.
- Encrypted traffic limits deep payload inspection but does not eliminate metadata analysis.

For these reasons, detection effectiveness depends on proper baseline analysis and careful rule calibration to minimize false positives while preserving sensitivity to malicious behavior. The detection framework described in this chapter forms the theoretical foundation for the experimental validation phase presented in the next chapter, where custom IDS rules are implemented and evaluated in a controlled LAN environment [11][12][22].

5.1 Experimental Environment Setup

This chapter presents the practical implementation and experimental validation of the proposed network-based intrusion detection approach for IP camera environments. The experiments were conducted within an isolated Local Area Network (LAN) designed to simulate realistic surveillance deployment conditions while maintaining a controlled and authorized testing environment. The experimental setup consisted of multiple interconnected devices operating within the same subnet. A Kali Linux machine was configured as the attacker system and used to perform ONVIF enumeration, RTSP observation, port scanning, and ARP spoofing attacks. In addition, a Hikvision IP camera was deployed as the target surveillance device supporting ONVIF and RTSP services.

A separate client device was also included to simulate legitimate network communication between users and the camera system. Furthermore, the network environment included monitoring and detection components designed to analyze traffic behavior during both normal and attack conditions. Wireshark was utilized to capture and inspect packets generated during the experiments, while Suricata was configured as a Network Intrusion Detection System (NIDS) to monitor suspicious activity and generate real-time alerts based on custom detection rules. In addition, Fail2ban was integrated with Suricata logs to provide automated blocking functionality against repeated suspicious behavior. The experiments focused specifically on LAN-based reconnaissance and interception activities without targeting external Internet services or exploiting firmware vulnerabilities. All attack simulations were performed for educational and defensive research purposes only[3][12][22].

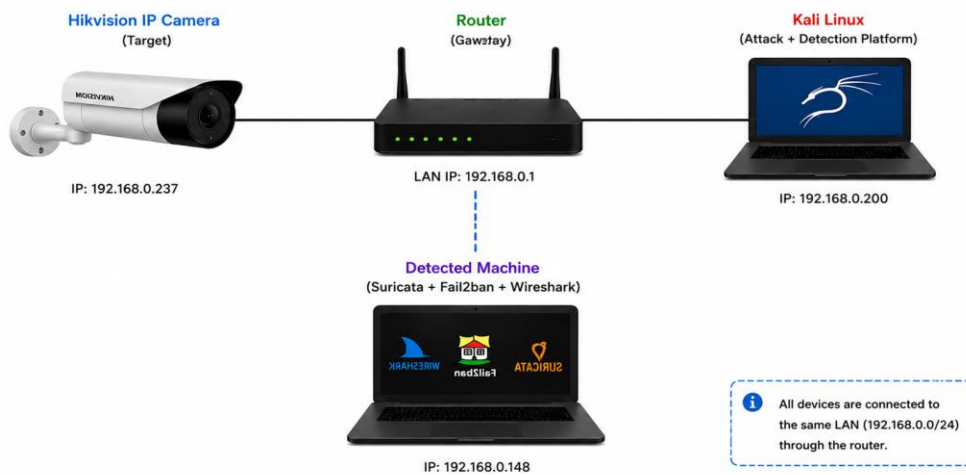
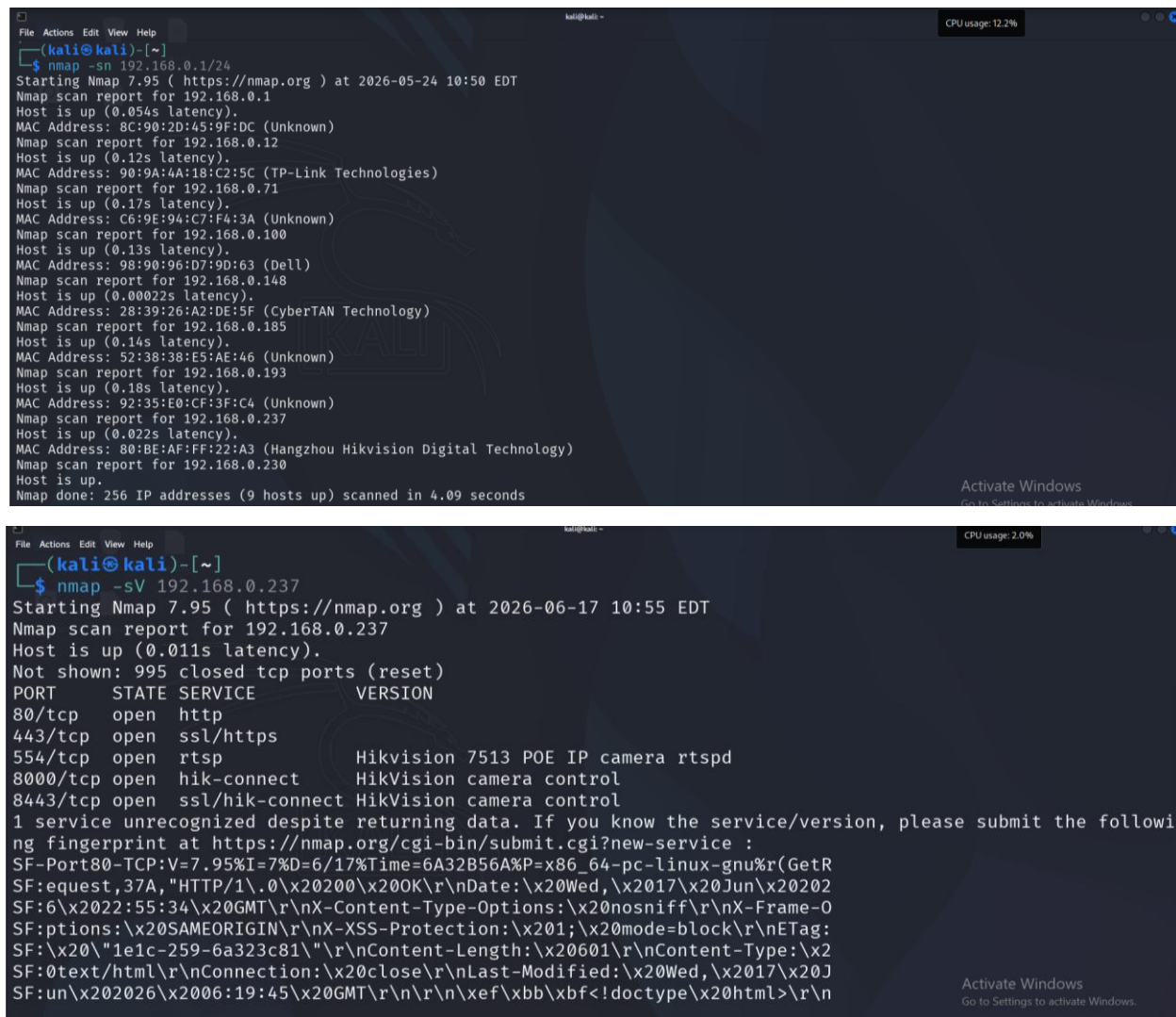


Figure 5.1: Experimental LAN Environment for IP Camera Security Testing

5.2 Initial Network Reconnaissance Using Nmap

Initial network reconnaissance was conducted using Nmap to identify active hosts and accessible services within the isolated LAN environment. The scanning process assisted in verifying network connectivity between the attacker system and the IP camera before performing further traffic analysis and intrusion detection experiments. The obtained results confirmed that the target IP camera was reachable within the local network and exposed RTSP-related communication services associated with real-time video streaming functionality. This phase provided important preliminary information regarding accessible ports and service availability, supporting the subsequent stages of traffic monitoring, RTSP analysis, and IDS-based detection experiments conducted during the project [11][12].



```
(kali@kali)-[~]
$ nmap -sn 192.168.0.1/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-24 10:50 EDT
Nmap scan report for 192.168.0.1
Host is up (0.054s latency).
MAC Address: 8C:90:2D:45:9F:DC (Unknown)
Nmap scan report for 192.168.0.12
Host is up (0.12s latency).
MAC Address: 90:9A:4A:18:C2:5C (TP-Link Technologies)
Nmap scan report for 192.168.0.71
Host is up (0.17s latency).
MAC Address: C6:9E:94:C7:F4:3A (Unknown)
Nmap scan report for 192.168.0.100
Host is up (0.13s latency).
MAC Address: 98:90:96:D7:9D:63 (Dell)
Nmap scan report for 192.168.0.148
Host is up (0.00022s latency).
MAC Address: 28:39:26:A2:DE:5F (CyberTAN Technology)
Nmap scan report for 192.168.0.185
Host is up (0.14s latency).
MAC Address: 52:38:38:E5:AE:46 (Unknown)
Nmap scan report for 192.168.0.193
Host is up (0.18s latency).
MAC Address: 92:35:E0:CF:3F:C4 (Unknown)
Nmap scan report for 192.168.0.237
Host is up (0.022s latency).
MAC Address: 80:BE:AF:FF:22:A3 (Hangzhou Hikvision Digital Technology)
Nmap scan report for 192.168.0.230
Host is up.
Nmap done: 256 IP addresses (9 hosts up) scanned in 4.09 seconds

(kali@kali)-[~]
$ nmap -sV 192.168.0.237
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-17 10:55 EDT
Nmap scan report for 192.168.0.237
Host is up (0.011s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
80/tcp    open  http
443/tcp   open  ssl/https
554/tcp   open  rtsp         Hikvision 7513 POE IP camera rtspd
8000/tcp  open  hik-connect  HikVision camera control
8443/tcp  open  ssl/hik-connect HikVision camera control
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port80-TCP:V=7.95I=7%D=6/17Time=6A32B56A%P=x86_64-pc-linux-gnu%r(GetR
SF:equest,37A,"HTTP/1\..0\x20200\x200K\r\nDate:\x20Wed,\x2017\x20Jun\x20202
SF:6\x2022:55:34\x20GMT\r\nX-Content-Type-Options:\x20nosniff\r\nX-Frame-0
SF:ptions:\x20SAMEORIGIN\r\nX-XSS-Protection:\x201;\x20mode=block\r\nETag:
SF:\x20"1e1c-259-6a323c81"\r\nContent-Length:\x20601\r\nContent-Type:\x2
SF:0text/html\r\nConnection:\x20close\r\nLast-Modified:\x20Wed,\x2017\x20J
SF:un\x202026\x2006:19:45\x20GMT\r\n\r\n\xef\xbb\xbf<!doctype\x20html>\r\n
```

Figure 5.2: Nmap Host Discovery and Service Detection Results Within the Experimental LAN Environment

5.3 ONVIF Enumeration Experiment

The first experimental phase focused on ONVIF-based device discovery and enumeration within the local network. Since ONVIF relies on WS-Discovery multicast communication, the attacker system was able to send discovery requests to identify active IP camera devices within the subnet. During the experiment, the attacker machine transmitted WS-Discovery Probe messages over UDP multicast address 239.255.255.250 on port 3702. The IP camera responded with ProbeMatch messages containing metadata related to the device configuration and supported services. The responses revealed information including device IP address, ONVIF service endpoints, and available communication services. The captured traffic demonstrated that ONVIF enumeration could be performed without requiring authentication or direct interaction with the camera management interface[6][7].

This behavior confirms that ONVIF discovery mechanisms may unintentionally expose device information to unauthorized hosts connected to the same LAN environment. Wireshark packet analysis showed repeated multicast discovery traffic and structured SOAP-based communication exchanged between the attacker system and the camera device. These traffic patterns provided observable indicators for network-based intrusion detection[11][12].

The screenshot displays a Kali Linux terminal window and a Wireshark packet capture. The terminal shows the execution of the `nmmap --script broadcast-wsdd-discover` command, which sends a WS-Discovery Probe to the multicast address 239.255.255.250 on port 3702. The output shows a successful discovery of a device at IP 192.168.0.237, with a message ID of 60ed0000-d3db-11b5-84b9-80b-eafff22a3. The device's ONVIF service endpoint is listed as `http://[fe80::82be:afff:feff:22a3]/onvif/device_service`.

The Wireshark packet capture shows the corresponding network traffic. The first packet is a WS-Discovery Probe (UDP) sent from the attacker (192.168.0.230) to the multicast address (239.255.255.250) on port 3702. The second packet is a WS-Discovery ProbeMatch (UDP) received from the IP camera (192.168.0.237) on port 3702. The packet details show the SOAP message structure, including the `wsdd:ProbeMatch` element, which contains the device's IP address, ONVIF service endpoints, and supported services.

Below the Wireshark capture, a Python script is shown that uses the `onvif` library to interact with the discovered device. The script creates an `ONVIFCamera` object for the IP camera and uses the `create_devicemgmt_service()` method to retrieve device information. The output of the script shows the following device details:

```
{
  'Manufacturer': 'HIKVISION',
  'Model': 'DS-2CD2T67G2H-LI',
  'FirmwareVersion': 'V5.7.14',
  'SerialNumber': 'DS-2CD2T67G2H-LI20231209AAWRAX7521103',
  'HardwareId': '88'
}
```

Figure 5.3: ONVIF Device Discovery and Enumeration Results

5.4 RTSP Traffic Observation

Following the ONVIF discovery phase, RTSP communication was analyzed to observe streaming-related traffic generated by the IP camera. The experiment aimed to examine how streaming sessions could be identified and monitored at the network level. The VideoLAN Client (VLC) application was used to test RTSP stream accessibility from the camera device.

During stream initialization, several RTSP request methods such as OPTIONS, DESCRIBE, SETUP, and PLAY were observed within the captured network traffic. Wireshark analysis demonstrated that RTSP communication generated identifiable TCP traffic directed toward port 554, which is commonly associated with real-time streaming services. Even when full stream access was restricted, metadata related to session establishment and communication behavior remained observable within packet captures. The experiment confirmed that RTSP traffic patterns can provide valuable indicators for monitoring unauthorized stream access attempts and abnormal communication activity within IP camera environments [6][11].

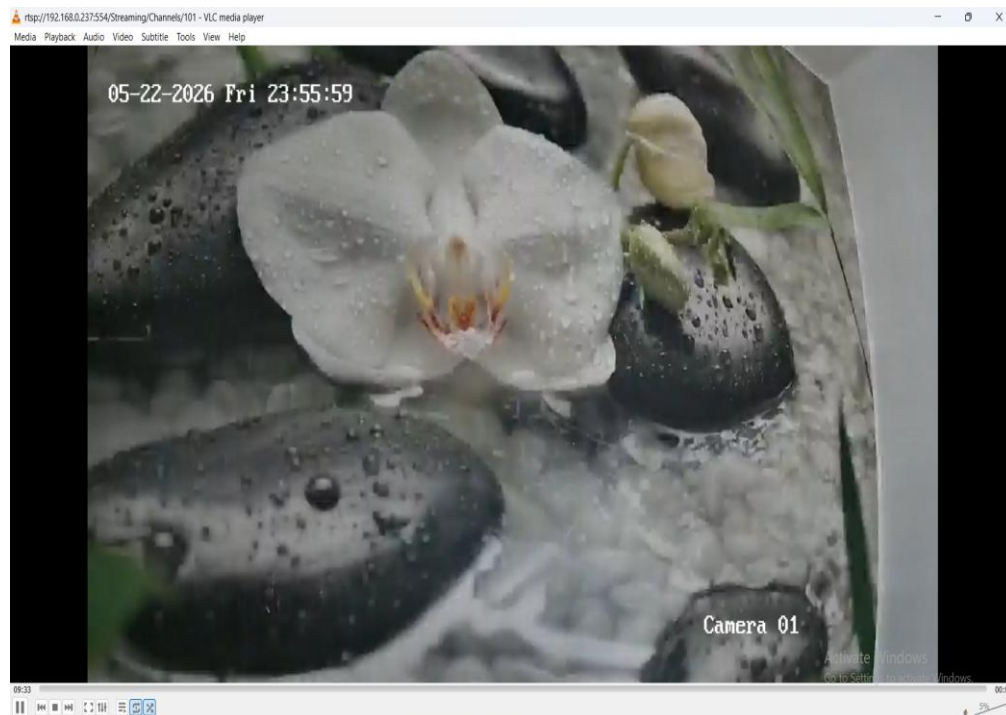


Figure 5.4: RTSP Communication Observed During Stream Initialization

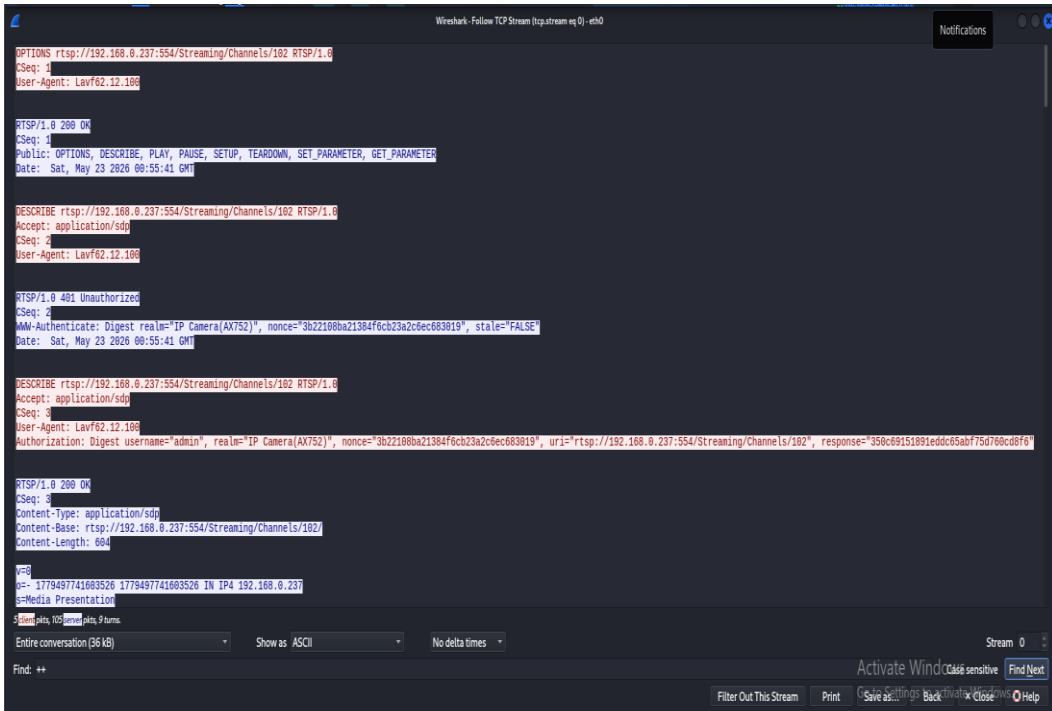


Figure 5.5: RTSP Session Establishment and Command Exchange Captured Using Wireshark

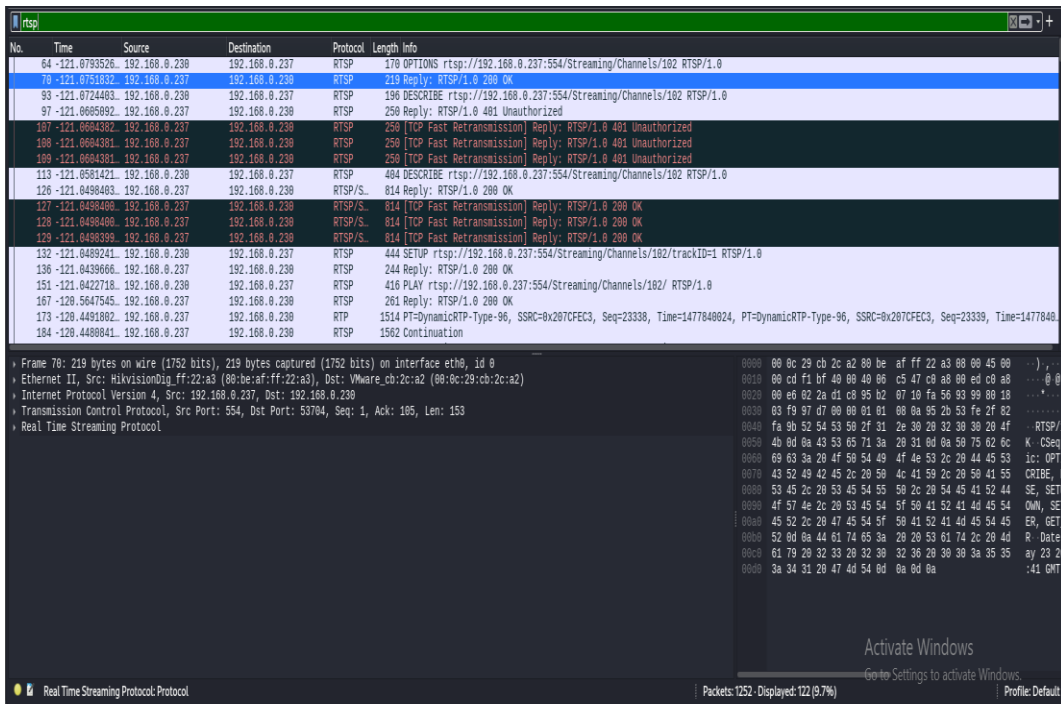


Figure 5.6: RTSP Packet Traffic and Response Analysis Observed in Wireshark

5.5 ARP Spoofing Simulation

The second major attack scenario involved ARP spoofing within the local network environment. The objective of this experiment was to demonstrate how protocol-level weaknesses in ARP communication could enable traffic interception without exploiting firmware vulnerabilities. During the experiment, the attacker system transmitted forged ARP reply packets to both the IP camera and the network gateway. These forged packets falsely associated the attacker's MAC address with legitimate device IP addresses, allowing the attacker machine to position itself as a Man-in-the-Middle (MITM) between communicating systems [21].

As the poisoning process continued, abnormal ARP traffic patterns became visible within Wireshark captures. Repeated unsolicited ARP replies and inconsistent IP-to-MAC mappings were observed during the attack period. Furthermore, duplicate ARP-related behavior appeared as the attacker continuously refreshed spoofed entries to maintain traffic interception [21].

The experiment demonstrated that ARP spoofing attacks can be executed entirely within local network environments using protocol manipulation techniques rather than direct exploitation methods. These characteristics make ARP poisoning a realistic threat in many IP camera deployments lacking internal monitoring and segmentation controls [5][21].

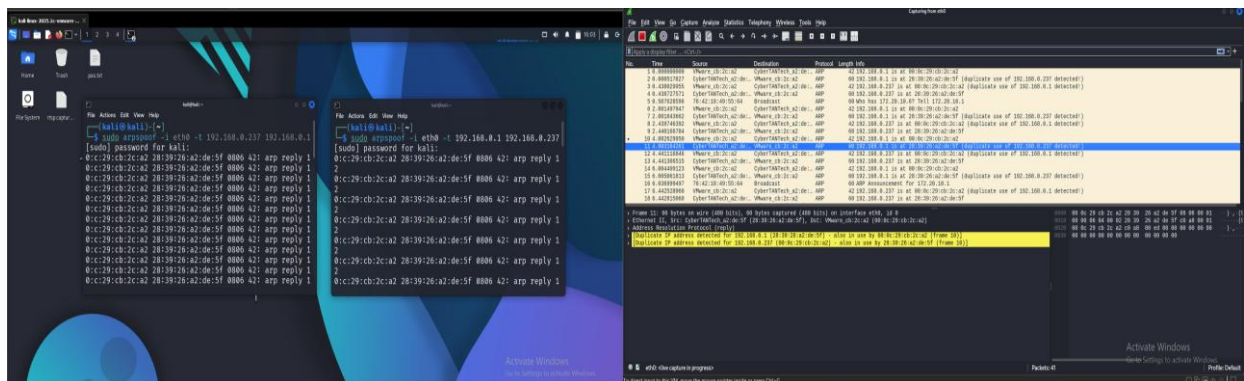


Figure 5.7: ARP Spoofing Activity and Abnormal ARP Traffic Patterns

5.6 IDS Rule Implementation Using Suricata

To detect suspicious network behavior generated during the experiments, Suricata was configured as a Network Intrusion Detection System (NIDS). Custom detection rules [22] were developed to identify traffic indicators associated with ONVIF discovery activity, RTSP access attempts, ICMP probing, and potential port scanning behavior. The implemented rules focused on behavioral detection rather than exploit signatures. For example, repeated RTSP communication toward port

554 and excessive ICMP traffic were monitored as potential indicators of reconnaissance activity. Similarly, abnormal traffic directed toward camera-related services generated alerts within the Suricata logging system. During testing, Suricata successfully generated real-time alerts recorded inside the fast.log file [22].

The alerts included information such as source IP addresses, protocol types, destination ports, and alert classifications. Multiple attack-related events including RTSP access attempts, ICMP probing, and port scan activity were successfully detected during the experiments. The generated alerts confirmed that network-based behavioral monitoring can provide effective visibility into suspicious LAN activity targeting IP camera systems [11][22].

```

(kali@kali) ~
$ sudo tail -f /var/log/suricata/fast.log
[sudo] password for kali:
05/18/2026-15:08:40.845276  [**] [1:1000004:1] Possible Port Scan [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:47974 → 1
192.168.0.237:445
05/18/2026-15:08:40.913814  [**] [1:1000001:1] RTSP Access Detected [*]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:47974 →
192.168.0.237:554
05/18/2026-15:22:46.890618  [**] [1:1000001:1] RTSP Access Detected [*]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:38889 →
192.168.0.237:554
05/18/2026-15:23:55.955576  [**] [1:1000004:1] Possible Port Scan [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:36707 → 1
192.168.0.237:143
05/18/2026-15:23:55.958581  [**] [1:1000001:1] RTSP Access Detected [*]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:36707 →
192.168.0.237:554
05/18/2026-15:24:00.226705  [**] [1:1000001:1] RTSP Access Detected [*]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:53433 →
192.168.0.237:554
05/18/2026-15:31:48.452609  [**] [1:1000004:1] Possible Port Scan [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:45922 → 1
192.168.0.237:113
05/18/2026-15:31:48.462797  [**] [1:1000001:1] RTSP Access Detected [*]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:45922 →
192.168.0.237:554
05/18/2026-15:31:57.185921  [**] [1:1000001:1] RTSP Access Detected [*]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:62424 →
192.168.0.237:554
05/18/2026-15:31:57.185820  [**] [1:1000004:1] Possible Port Scan [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.230:62424 → 1

```

Figure 5.8: Suricata Fast Log Alerts Generated During RTSP and Port Scanning Activities

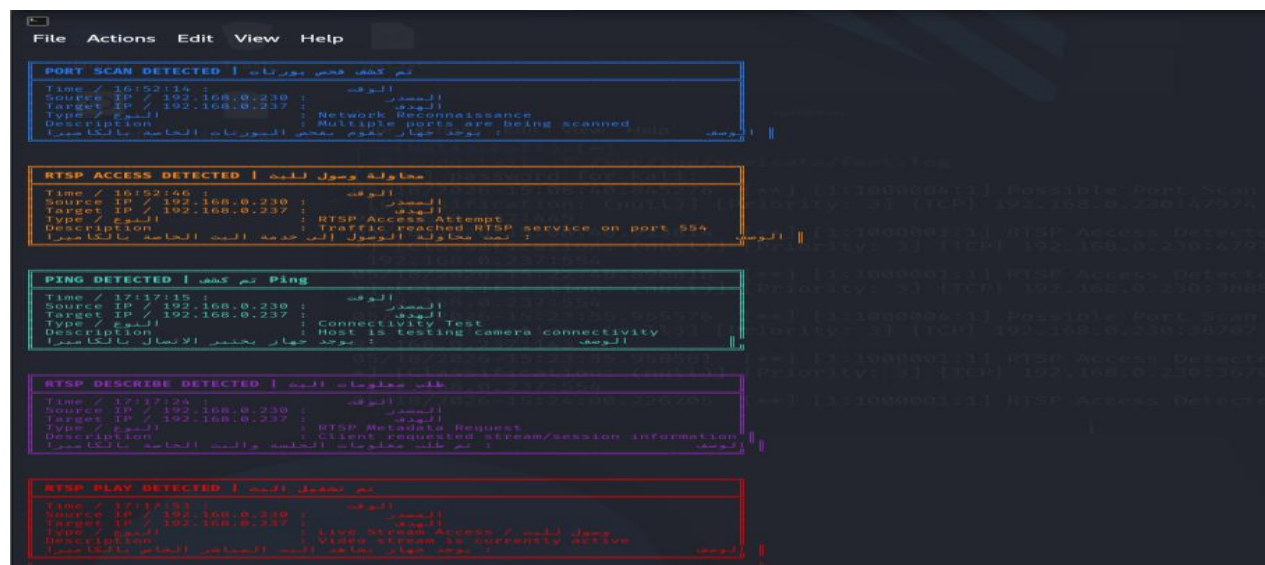
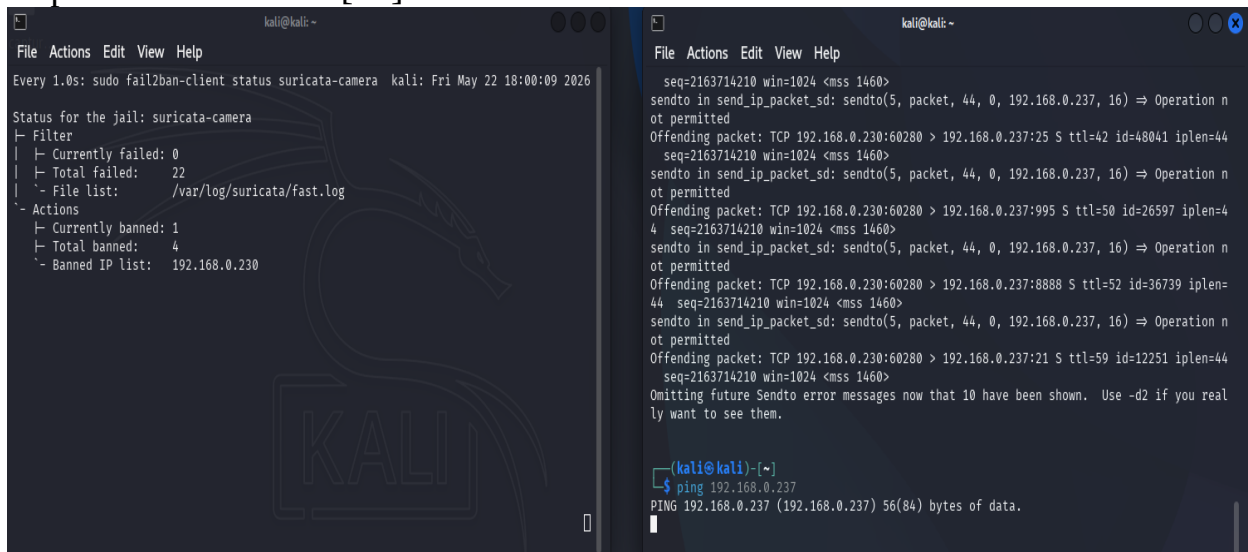


Figure 5.9: Python-Based Security Monitoring Interface Displaying Real-Time Detection Alerts

5.7 Automated Blocking Using Fail2ban

To enhance the defensive capabilities of the monitoring system, Fail2ban was integrated with Suricata alert logs to provide automated response functionality. The objective of this phase was to demonstrate how repeated suspicious behavior could trigger temporary network blocking actions. Fail2ban was configured to continuously monitor Suricata alert entries stored within the fast.log file. Detection thresholds including retry count, monitoring interval, and ban duration were defined to identify repeated malicious behavior originating from the same source IP address. When repeated attack-related alerts exceeded the configured threshold, Fail2ban automatically generated blocking actions using iptables firewall rules.

The attacker IP address was temporarily banned from communicating with the monitored environment for a predefined time period. System logs confirmed successful detection and blocking operations. Ban and unban events were recorded within Fail2ban logs, demonstrating automated IPS-like behavior capable of responding dynamically to suspicious activity. The integration between Suricata, Fail2ban, and iptables significantly improved the practical defensive capabilities of the experimental environment by combining traffic detection with automated response mechanisms [22].



The image shows two terminal windows from a Kali Linux system. The left window displays the output of the command 'sudo fail2ban-client status suricata-camera'. It shows the status for the jail 'suricata-camera', including the filter 'suricata-fastlog', the file list '/var/log/suricata/fast.log', and the actions 'iptables'. It also shows the current failed count (0), total failed count (22), and the banned IP list (192.168.0.237). The right window shows the output of the command 'fail2ban-client log -f -n 10'. It displays a series of log entries showing the detection of a SYN flood attack from 192.168.0.237, including the packet details and the action taken (ban).

```
kali@kali: ~  
File Actions Edit View Help  
Every 1.0s: sudo fail2ban-client status suricata-camera kali: Fri May 22 18:00:09 2026  
  
Status for the jail: suricata-camera  
├─ Filter  
│ └─ Currently failed: 0  
│ └─ Total failed: 22  
│ └─ File list: /var/log/suricata/fast.log  
└─ Actions  
  └─ Currently banned: 1  
  └─ Total banned: 4  
  └─ Banned IP list: 192.168.0.237  
  
kali@kali: ~  
File Actions Edit View Help  
seq=2163714210 win=1024 <mss 1460>  
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 192.168.0.237, 16) => Operation not permitted  
Offending packet: TCP 192.168.0.230:60280 > 192.168.0.237:25 S ttl=42 id=48041 iphlen=44  
seq=2163714210 win=1024 <mss 1460>  
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 192.168.0.237, 16) => Operation not permitted  
Offending packet: TCP 192.168.0.230:60280 > 192.168.0.237:995 S ttl=50 id=26597 iphlen=44  
seq=2163714210 win=1024 <mss 1460>  
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 192.168.0.237, 16) => Operation not permitted  
Offending packet: TCP 192.168.0.230:60280 > 192.168.0.237:8888 S ttl=52 id=36739 iphlen=44  
seq=2163714210 win=1024 <mss 1460>  
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 192.168.0.237, 16) => Operation not permitted  
Offending packet: TCP 192.168.0.230:60280 > 192.168.0.237:21 S ttl=59 id=12251 iphlen=44  
seq=2163714210 win=1024 <mss 1460>  
Omitting future Sendto error messages now that 10 have been shown. Use -d2 if you really want to see them.  
  
(kali@kali)~  
$ ping 192.168.0.237  
PING 192.168.0.237 (192.168.0.237) 56(84) bytes of data.
```

Figure 5.10: Automated Blocking Response

5.8 Traffic Monitoring and Detection Results

The final experimental results demonstrated that LAN-based attacks against IP camera systems produce observable traffic characteristics that can be detected using network-level monitoring techniques. ONVIF enumeration generated repeated WS-Discovery multicast traffic that revealed identifiable reconnaissance behavior. RTSP observation experiments demonstrated visible streaming communication

patterns associated with camera access attempts. Furthermore, ARP spoofing attacks produced abnormal ARP communication behavior including unsolicited replies and repeated address mapping changes.

The implemented Suricata detection rules successfully identified multiple suspicious activities during the experiments, while Fail2ban provided automated blocking functionality against repeated attack attempts. This combination demonstrated the effectiveness of behavioral monitoring and automated response mechanisms in improving visibility and defensive capabilities within IP camera LAN environments. The experimental findings confirm that protocol-based attacks targeting IoT surveillance systems can be detected through traffic analysis even when no firmware exploitation occurs. These results support the importance of network-based intrusion detection strategies for securing modern IP camera deployments [11][12][22].

Table 5.1: Summary of Detected Activities and Defensive Responses in the Experimental Environment

 Activity	 Detection Result	 Defensive Response
 ONVIF Enumeration	 Detected Successfully	 Alert Generated
 RTSP Access Attempt	 Detected Successfully	 Logged by Suricata
 ICMP Reconnaissance	 Detected Successfully	 Temporary Ban Applied
 Port Scanning Activity	 Detected Successfully	 Alert and Blocking
 ARP Spoofing Behavior	 Observed Through Traffic Analysis	 Logged for Analysis

6.1 Conclusion

This project evaluated the security impact of LAN-based attacks targeting IP camera environments operating within local area networks. The practical experiments focused on ONVIF enumeration, RTSP traffic observation, ARP spoofing attacks, and network-based intrusion detection techniques. A controlled and isolated experimental environment was implemented using a real Hikvision IP camera, Kali Linux attacker system, Wireshark traffic analysis, Suricata IDS monitoring, and automated blocking mechanisms. The experimental results demonstrated that ONVIF discovery services can expose identifiable device information through WS-Discovery multicast communication. The captured traffic confirmed that ONVIF-enabled devices may respond to discovery requests without requiring prior authentication, allowing reconnaissance activity to be performed inside the local network environment [6][7].

Furthermore, RTSP communication analysis revealed observable session establishment behavior during streaming initialization. Several RTSP methods including OPTIONS, DESCRIBE, SETUP, and PLAY were identified through packet analysis, demonstrating that communication metadata and session-related information remain observable at the network level even when direct stream access is restricted [6][11].

The ARP spoofing experiments also demonstrated that protocol-level weaknesses within LAN environments can enable traffic interception without exploiting firmware vulnerabilities. Wireshark traffic analysis showed repeated unsolicited ARP replies and abnormal IP-to-MAC mapping behavior associated with Man-in-the-Middle positioning attempts [5][21].

In addition, the deployment of Suricata as a Network Intrusion Detection System successfully generated alerts associated with suspicious RTSP access attempts, ICMP probing, and port scanning activity. The integration between Suricata, Fail2ban, and iptables further improved the defensive capabilities of the environment by enabling automated temporary blocking actions against repeated suspicious behavior. Overall, the project confirmed that behavioral traffic analysis and network-based monitoring approaches can provide effective visibility into suspicious communication patterns targeting IP camera systems. The obtained results emphasize the importance of IDS and IPS technologies for improving the security posture of IoT-based surveillance environments operating within local area networks [11][12][22].

The findings obtained throughout this project highlight the importance of continuous monitoring, behavioral detection strategies, and automated defensive mechanisms for improving the security of modern IP camera deployments and IoT surveillance infrastructures. The integration of network traffic analysis and intrusion detection techniques provides an effective approach for identifying suspicious activities and enhancing the overall security posture of LAN-based surveillance environments[1][11][12][22].

6.2 Limitations of the Study

Despite the successful implementation of the experimental environment, several limitations were identified during the project:

- The experiments were conducted within a small-scale isolated LAN environment containing a limited number of devices.
- The study focused specifically on LAN-based reconnaissance and interception attacks rather than Internet-facing exploitation scenarios.
- The implemented IDS rules were customized for the experimental environment and may require additional tuning for large enterprise deployments.
- The project focused primarily on traffic-based behavioral monitoring without performing firmware exploitation or advanced vulnerability assessment techniques.
- The experiments were conducted under controlled educational conditions using authorized devices and simulated attack scenarios.

6.3 Future Work

Several improvements and extensions may enhance the effectiveness of future research related to IP camera security and IoT intrusion detection systems:

- Implement machine learning techniques for anomaly-based detection of suspicious IP camera traffic[12][13].
- Expand the monitoring environment to support larger enterprise surveillance infrastructures.
- Integrate centralized Security Information and Event Management (SIEM) platforms for advanced event correlation and alert management.
- Perform encrypted RTSP traffic analysis to improve visibility into secure streaming environments.

- Develop automated network segmentation mechanisms for isolating compromised surveillance devices.
- Extend detection capabilities to support additional IoT communication protocols and smart surveillance technologies.
- Investigate firmware analysis and embedded vulnerability assessment techniques for IP camera devices [1][12][20][23].

References

- [1] European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024*, ENISA, 2024.
- [2] M. Fagan, K. Megas, K. Scarfone, and M. Smith, *IoT Device Cybersecurity Capability Core Baseline*, NISTIR 8259A, National Institute of Standards and Technology (NIST), 2020.
- [3] National Institute of Standards and Technology (NIST), *IoT Device Cybersecurity Guidance for the Federal Government: Establishing IoT Device Cybersecurity Requirements*, NIST SP 800-213, 2021.
- [4] European Telecommunications Standards Institute (ETSI), *Cyber Security for Consumer Internet of Things: Baseline Requirements*, ETSI EN 303 645, 2020.
- [5] Open Web Application Security Project (OWASP), *OWASP Internet of Things Security Guidance*, OWASP Foundation, 2024.
- [6] Open Network Video Interface Forum (ONVIF), *ONVIF Core Specification*, ONVIF.
- [7] Open Network Video Interface Forum (ONVIF), *ONVIF Profile Specifications*, ONVIF.
- [8] N. Neshenko, E. Bou-Harb, J. Crichigno, G. Kaddoum, and N. Ghani, **“Demystifying IoT Security: An Exhaustive Survey on IoT Vulnerabilities and a First Empirical Look on Internet-Scale IoT Exploitations,”** *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2702–2733, 2019.
- [9] P. Mishra and S. Pandya, **“Internet of Things Applications, Security Challenges, and Future Trends,”** *IEEE Access*, vol. 9, pp. 117820–117842, 2021.
- [10] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, **“Network Intrusion Detection for IoT Security Based on Learning Techniques,”** *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2671–2701, 2019.
- [11] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, **“Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges,”** *Cybersecurity*, vol. 2, no. 1, 2019.

- [12] A. Buczak and E. Guven, “**A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection,**” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [13] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, “**A Survey of Network-Based Intrusion Detection Data Sets,**” *Computers & Security*, vol. 86, pp. 147–167, 2019.
- [14] S. Sicari, A. Rizzardi, L. Grieco, and A. Coen-Porisini, “**Security, Privacy and Trust in Internet of Things: The Road Ahead,**” *Computer Networks*, vol. 76, pp. 146–164, 2015.
- [15] S. Zeadally, F. Siddiqui, and Z. Baig, “**25 Years of Internet of Things: Evolution, Enabling Technologies, and Security Challenges,**” *IEEE Communications Surveys & Tutorials*, 2021.
- [16] A. Iacovazzi, H. Wang, I. Butun, and S. Raza, “**Towards Cyber Threat Intelligence for the IoT,**” 2024.
- [17] Z. Alwaisi and S. Soderi, “**Towards Robust IoT Defense: Comparative Statistics of Attack Detection in Resource-Constrained Scenarios,**” 2024.
- [18] Cisco Systems, *Network Segmentation and Security Best Practices*, Cisco Documentation.
- [19] National Institute of Standards and Technology (NIST), *Cybersecurity Framework (CSF) 2.0*, NIST, 2024.
- [20] P. Biondi, S. Bognanni, and G. Bella, “**Vulnerability Assessment and Penetration Testing on IP Cameras,**” arXiv preprint arXiv:2202.06597, 2022.
- [21] S. Gangan, “**A Review of Man-in-the-Middle Attacks,**” arXiv preprint arXiv:1504.02115, 2015.
- [22] Open Information Security Foundation (OISF), *Suricata User Guide*, Suricata Documentation, 2025.